

```

const CACHE_NAME = 'bathroomreport-v59';
const APP_SHELL = [
  './',
  './index.html',
  './styles.css',
  './shell.css',
  './logo.png',
  './icon-x.png',
  './icon-cashapp.png',
  './firebase.js',
  './stewarts-locations.js',
  './cumberland-farms-locations.js',
  './wawa-locations.js',
  './fastrac-locations.js',
  './alltownfresh-locations.js',
  './byrne-dairy-locations.js',
  './parkers-locations.js',
  './sheetz-locations.js',
  './racetrac-locations.js',
  './pilot-locations.js',
  './maverik-locations.js',
  './app.js',
  './manifest.webmanifest'
];

self.addEventListener('install', event => {
  // Cache each file independently rather than cache.addAll (which fails the ENTI
  // install if even one URL 404s – this is exactly what happened when locations.
  // was renamed during the multi-chain refactor, silently breaking every update
  event.waitUntil(
    caches.open(CACHE_NAME).then(cache =>
      Promise.all(APP_SHELL.map(url =>
        cache.add(url).catch(err => console.warn('sw precache skipped (not fatal)
      ))
    )
  );
  self.skipWaiting();
});

self.addEventListener('activate', event => {
  event.waitUntil(
    caches.keys().then(keys => Promise.all(keys.filter(key => key !== CACHE_NAME)
  );
});

```

```

    self.clients.claim();
  });

self.addEventListener('fetch', event => {
  if(event.request.method !== 'GET') return;
  const url = new URL(event.request.url);
  if(url.origin !== self.location.origin) return;

  const path = url.pathname;

  // CODE + SHELL → network-first. The app's own code (app.js, the stylesheets, f
  // and every navigation always try the network first, so a new deploy takes eff
  // very next load instead of waiting for a cache cycle. Falls back to cache whe
  // This is what prevents stale old code (e.g. a removed bulk read) from lingeri
  const isCodeShell = event.request.mode === 'navigate'
    || path === '/'
    || /\.(index\.html|app\.js|shell\.css|styles\.css|firebase\.js)$/.test(path);

  if(isCodeShell){
    event.respondWith(
      caches.open(CACHE_NAME).then(cache =>
        fetch(event.request)
          .then(response => {
            if(response && response.status === 200) cache.put(event.request, resp
            return response;
          })
          .catch(() => cache.match(event.request).then(c => c || cache.match('./i
        )
      );
    return;
  }

  // EVERYTHING ELSE (big chain-data JS, images, manifest) → stale-while-revalida
  // cached copy instantly so repeat visits don't re-download ~1 MB of data, and
  // the background for next time.
  event.respondWith(
    caches.open(CACHE_NAME).then(cache =>
      cache.match(event.request).then(cached => {
        const networkFetch = fetch(event.request)
          .then(response => {
            if(response && response.status === 200) cache.put(event.request, resp
            return response;
          })
          .catch(() => cached || caches.match('./index.html'));
        return cached || networkFetch;
      })
    )
  );
}

```

```
)  
);  
});
```